

Differences Between Transactional (OLTP) and Analytical (OLAP) Systems Explained

 systemdesignschool.io/fundamentals/oltp-vs-olap



OLTP (Transaction) or OLAP (Analytics)?

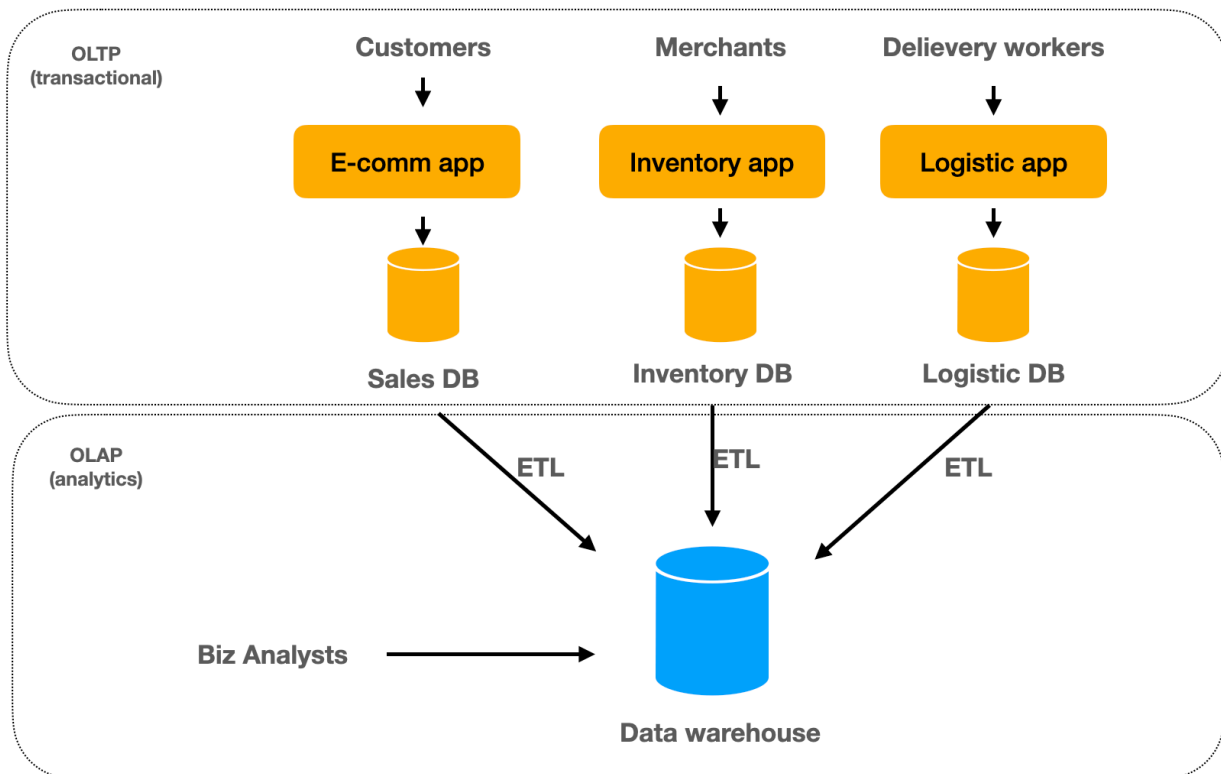
We've covered databases for applications: storing users, processing orders, managing content. But organizations also need to analyze data. Different access patterns require different database designs.

Two Different Workloads

Transactional databases (OLTP) serve applications. They handle user interactions in real-time. Insert an order. Update inventory. Delete a comment. Each operation touches a few rows. Thousands of these operations happen per second.

Analytical databases (OLAP) serve business intelligence. They answer aggregate questions. What's total revenue by region? Which products sell together? How many users signed up each month? Each query scans millions of rows. Fewer queries run, but each one is expensive.

Production systems use both. OLTP databases record transactions. An ETL pipeline extracts this data, transforms it, and loads it into OLAP systems for analysis.



OLTP and OLAP example diagram

Transactional (OLTP) Databases

OLTP databases optimize for high throughput of small operations. Each query touches a few rows using a primary key or index. An e-commerce site looks up a user by ID, updates their cart, and commits the transaction. This happens thousands of times per second.

Indexes are crucial. OLTP queries need microsecond latency. B-tree indexes enable fast lookups by key. Without indexes, queries would scan entire tables. User experience suffers when queries take seconds instead of milliseconds.

Storage models vary. Traditional databases like MySQL and PostgreSQL use B-trees and update data in place. Modern databases like Cassandra and HBase use log-structured merge trees. They append writes to logs and periodically compact them. Both approaches optimize for different trade-offs between read and write performance.

Analytical (OLAP) Databases

OLAP databases optimize for throughput of large scans. Each query reads millions of rows. Calculate total sales by region? Scan the entire orders table. Group by region and sum amounts. Queries run less frequently but process more data.

Indexes matter less. Analytical queries scan many rows sequentially. An index on region helps find rows for one region. But the query needs all regions. The database reads most of the table anyway. Sequential scans become faster than index lookups.

Column-oriented storage is essential. Analytical queries select specific columns from wide tables. "Sum the revenue column" doesn't need product names or descriptions. Column stores keep each column in a separate file. Read only the revenue column. This reduces disk I/O dramatically.

Compression saves space. Column stores compress better than row stores. The revenue column contains only numbers. Similar values appear together. Compression algorithms achieve high ratios. Amazon Redshift and Google BigQuery use column stores for this reason.

Analytical frameworks process data in parallel. MapReduce, Hadoop, and Spark split queries across multiple servers. Each server processes a partition. Results merge at the end. This parallelism enables querying petabytes of data.

Comparison

Feature	OLTP (Online Transaction Processing)	OLAP (Online Analytical Processing)
Primary Use Case	Transactional processing, user-facing	Internal analytical processing, data warehousing
Main Read pattern	Small number of records per query, typically using a key	Aggregation of a large number of records
Main Write pattern	Random-access, low-latency writes from user data	Bulk imports using ETL pipelines or event streams
Transactions	High frequency, small transactions	Low frequency, large analytical queries
Importance of Indexes	Crucial for efficient query execution	Less important due to sequential row scanning
Storage Models	Log-based (e.g., Cassandra, HBase) and Update-in-place (e.g., MySQL, PostgreSQL)	Column-oriented storage (e.g., Google Dremel, Amazon Redshift)
Data Models	Normalized data model (e.g., relational databases)	Denormalized data model (e.g., star, snowflake schema)
Data Encoding	Not a primary focus	Compact data encoding is essential
Analytical Frameworks	Not applicable	MapReduce, Hadoop, Spark
Examples	MySQL, PostgreSQL, Oracle, SQL Server, Cassandra, HBase	Google Dremel, Spark, Amazon Redshift, Cloudera

Feature	OLTP (Online Transaction Processing)	OLAP (Online Analytical Processing)
QPS (Queries per Second)	High	Low